📚 جزوه سیستم عامل | فصل ۸: بن‌بست (Deadlock)

📝 بخش ۳ و ۴: مشخصه‌های بن‌بست و روش‌های برخورد

● بخش ۳: مشخصه‌های بن‌بست (Deadlock Characterization)

در بخش قبل یاد گرفتیم بن‌بست چیست. حالا سوال این است: **دقیقاً چه زمانی بن‌بست رخ می‌دهد؟** طبق متن کتاب، برای وقوع بن‌بست، **هر ۴ شرط زیر باید به صورت همزمان** برقرار باشند. (اگر حتی یکی نباشد، بن‌بست نداریم).

۱. چهار شرط لازم (The 4 Necessary Conditions) ⚠

انحصار متقابل (Mutual Exclusion):

- حداقل یک منبع باید **غیرقابل اشتراک** باشد.
- یعنی در هر لحظه فقط یک نخ بتواند آن را بگیرد. اگر نخ دیگری آن را بخواهد، باید صبر کند.
- *مثال:* پرینتر یا CD-RW (نمیتوان همزمان دو چیز رایت کرد).

گرفتن و منتظر ماندن (Hold and Wait):

- یک نخ باید **حداقل یک منبع را گرفته باشد** (Hold) و همزمان **منتظر گرفتن منبع دیگری باشد** (Wait) که فعلاً دست نخ دیگری است.

نبود پس‌گیری (No Preemption):

- منابع را نمی‌توان به زور از نخ گرفت.
- منبع فقط زمانی آزاد می‌شود که نخی که آن را دارد، کارش تمام شود و **داوطلبانه** آن را رها کند.

انتظار چرخشی (Circular Wait):

- باید زنجیره‌ای از نخ‌ها وجود داشته باشد که هر کدام منتظر بعدی باشند.
- T_0 منتظر T_1 است، T_1 منتظر T_2 ... و T_n منتظر T_0 است. (یک دایره انتظار).

۲. گراف تخصیص منبع (Resource Allocation Graph - RAG) 📊

برای تحلیل دقیق‌تر بن‌بست، از یک گراف جهت‌دار استفاده می‌کنیم. این گراف بهترین ابزار برای حل سوالات امتحانی است.

اجزای گراف:

- **دایره (T):** نشان‌دهنده **نخ‌ها (Threads)** یا پردازه‌ها.
- **مستطیل (R):** نشان‌دهنده **انواع منابع (Resources)**.
  - *نقطه‌های داخل مستطیل:* تعداد نمونه‌های آن منبع (مثلاً اگر ۴ نقطه دارد یعنی ۴ پرینتر داریم).
- **یال درخواست ($T \rightarrow R$):** فلش از نخ به منبع. یعنی "من این منبع را می‌خواهم".
- **یال تخصیص ($R \rightarrow T$):** فلش از منبع به نخ. یعنی "این منبع فعلاً دست این نخ است".

تحلیل گراف (نکته طلایی برای تشخیص بن‌بست):

اگر گراف "دور" (Cycle) نداشته باشد:

- ✅ قطعاً بن‌بست **نداریم**.

اگر گراف "دور" داشته باشد:

- **حالت الف) منابع تک‌نمونه‌ای:** (هر مستطیل فقط یک نقطه داشته باشد)
  - ❌ قطعاً بن‌بست **داریم**. (در این حالت: وجود دور $\Leftrightarrow$ بن‌بست).
- **حالت ب) منابع چندنمونه‌ای:** (مستطیل‌ها چند نقطه داشته باشند)
  - ⚠️ **ممکن است** بن‌بست باشد، ممکن است نباشد.
  - *نکته:* اگر دور وجود دارد اما نخی خارج از دور وجود دارد که می‌تواند منبعی را آزاد کند و چرخه را بشکند، بن‌بست نیست.

● بخش ۴: روش‌های رسیدگی به بن‌بست (Methods for Handling Deadlocks)

به طور کلی سیستم‌عامل‌ها به ۳ روش با مشکل بن‌بست برخورد می‌کنند:

۱. نادیده گرفتن کل ماجرا (الگوریتم شترمرغ! 🐦)

- **روش:** فرض می‌کنیم بن‌بست هرگز رخ نمی‌دهد. اگر هم رخ داد، سیستم قفل می‌کند و کاربر مجبور می‌شود ریستارت کند.
- **کجا استفاده می‌شود؟** لینوکس (Linux)، ویندوز (Windows) و اکثر سیستم‌عامل‌های مدرن.
- **چرا؟** چون پیاده‌سازی راه‌حل‌های ضد بن‌بست پیچیده و پرهزینه است و بن‌بست‌ها خیلی کم (مثلاً ماهی یک بار) رخ می‌دهند. نمی‌صرفد سرعت سیستم را فدای آن کنیم.

۲. پیشگیری یا اجتناب (Prevention / Avoidance)

این روش‌ها تضمین می‌کنند که سیستم **هرگز** وارد بن‌بست نشود.

- **پیشگیری (Prevention):** قوانینی می‌گذاریم که حداقل یکی از آن ۴ شرط لازم (بخش ۳) نقض شود. (مثلاً اجازه ندهیم کسی همزمان منبع نگه دارد و منتظر باشد).
- **اجتناب (Avoidance):** سیستم اطلاعات قبلی می‌گیرد (مثلاً هر کس چقدر منبع می‌خواهد). قبل از دادن هر منبع، سیستم‌عامل چک می‌کند: "آیا اگر این را بدهم امن است؟". اگر خطرناک بود، نمی‌دهد (مثل **الگوریتم بانکدار**).

۳. تشخیص و ترمیم (Detection and Recovery)

روش: اجازه می‌دهیم بن‌بست رخ دهد. سپس:

1. سیستم با الگوریتمی وضعیت را بررسی می‌کند تا بن‌بست را **تشخیص** دهد.
2. با روش‌هایی (مثل بستن پردازه‌ها یا پس‌گیری منابع) وضعیت را **ترمیم** می‌کند.

کاربرد: بیشتر در سیستم‌های پایگاه داده (Database) استفاده می‌شود.

🌟 خلاصه نکات کنکوری (جمع‌بندی)

نکته تستی	توضیح کوتاه	مفهوم
همه باید **همزمان** باشند.	۴ تاست: انحصار متقابل، نگهداری و انتظار، عدم پس‌گیری، انتظار چرخشی.	**شرط لازم**
هرگز بن‌بست نیست.	هیچ چرخه‌ای در فلش‌ها نیست.	**گراف بدون دور**
قطعاً بن‌بست است.	منابع فقط ۱ عدد موجودی دارند.	**گراف با دور (تک‌نمونه)**
شاید بن‌بست باشد (بستگی به شرایط دارد).	منابع چند عدد موجودی دارند.	**گراف با دور (چندنمونه)**
فرض می‌کنند بن‌بست وجود نادیده گرفتن (Ignore).		**رویکرد**

ندارد. **ویندوز/لینوکس**